

Assurance Assignment: Hand-in

Role: SAMM specialist

Francesco Maria Fuligni | Group 13

1. Core

Security Testing (Verification)

Dedicated security test cases for known-risk areas, run regularly. The audit covered authorization, path traversal, login throttling and link access. For link access, the test fixture issued links for two users and checked that each link was a 40-character lowercase hexadecimal value. Requesting a valid issued link without logging in returned HTTP 200 and the PDF. Four unissued candidates—a random value, the issued link with its last character changed, its numeric value plus one, and an all-zero value—each returned HTTP 404 and no PDF. Each user's link returned only their own document and watermark secret. These checks cover the tested cases; they do not prove that every possible link is unpredictable. Pytest runs in CI for pull requests to `main` and pushes to `main` and `feat/**`.

Improvement

A regression test was added to check that a predictable SHA-1-derived link cannot retrieve a PDF. Extended negative-link tests to check that responses reveal no user secrets, recipients or filenames. See commit `fbae18a`.

Scorecard

Level 1	Level 2 – assigned	Level 3
Security tested only incidentally.	Dedicated security test cases for known-risk areas, run regularly.	Systematic testing, negative cases and tracked coverage.

Evidence: The assignment's Level 2 criterion is supported by dedicated tests for known-risk areas: authorization, path traversal, login throttling and link access. They run regularly in CI on pull requests to `main` and pushes to `main` and `feat/**`. This provides repeatable, application-specific security testing, consistent with SAMM's risk-based approach.

2. Further Work

Secure Build (Implementation)

Reviewed the Dockerfile, Python lockfile, CI workflow and Compose configuration to check whether the container build was tested in CI and used the pinned dependencies. Before the improvement, CI did not build the image, and the Dockerfile did not use the lockfile. The built image contained `cryptography 50.0.2`, while the lockfile pinned `50.0.1`. A manual image build and `rmap` import succeeded, and Compose configuration validation passed.

Improvement

Added CI checks to build the Docker image and detect dependency drift, and pinned the image and Python dependencies so the container and tests use consistent versions. The clean build matched all 26 pinned versions; dependency validation and `import rmap` passed. CI also passed Ruff and pytest (368 passed, 11 skipped, 2 xfailed). See CI run and commits `8a1073f`, `f0cd7ae`, and `f7e50c3`.

Scorecard

Level 1	Level 2 – assigned	Level 3
Manual, inconsistent builds.	Automated, reproducible builds; pinned dependencies.	Automated security checks as a build-pipeline gate.

Evidence: For the server/Python build, the assignment's Level 2 criterion is supported by a Docker build and import check in CI, tracked Python versions, and a passing dependency-drift check. Together, these show an automated, repeatable build with controlled dependencies, consistent with OWASP SAMM's Secure Build practice.

3. Hours

Self-reported hours.

	Reading	Audit	Improvement
Core	1 h	3 h	1 h
Further Work	30 min	3 h	1 h

Total: approximately 10 hours. Does not include Track 3 hours (4).